

USE CLAUDE CODE

How Claude remembers your project

Give Claude persistent instructions with CLAUDE.md files, and let Claude accumulate learnings automatically with auto memory.

Each Claude Code session begins with a fresh context window. Two mechanisms carry knowledge across sessions:

- **CLAUDE.md files:** instructions you write to give Claude persistent context
- **Auto memory:** notes Claude writes itself based on your corrections and preferences

This page covers how to:

- Write and organize CLAUDE.md files
- Scope rules to specific file types with `.claude/rules/`
- Configure auto memory so Claude takes notes automatically
- Troubleshoot when instructions aren't being followed

CLAUDE.md vs auto memory

Claude Code has two complementary memory systems. Both are loaded at the start of every conversation. Claude treats them as context, not enforced configuration. To block an action regardless of what Claude decides, use a PreToolUse hook instead. The more specific and concise your instructions, the more consistently Claude follows them.

	CLAUDE.md files	Auto memory
Who writes it	You	Claude
What it contains	Instructions and rules	Learnings and patterns
Scope	Project, user, or org	Per repository, shared across worktrees
Loaded into	Every session	Every session (first 200 lines or 25KB)
Use for	Coding standards, workflows, project	Build commands, debugging insights, preferences

CLAUDE.md files	Auto memory
architecture	Claude discovers

Use CLAUDE.md files when you want to guide Claude’s behavior. Auto memory lets Claude learn from your corrections without manual effort.

Subagents can also maintain their own auto memory. See [subagent configuration](#) for details.

CLAUDE.md files

CLAUDE.md files are markdown files that give Claude persistent instructions for a project, your personal workflow, or your entire organization. You write these files in plain text; Claude reads them at the start of every session.

When to add to CLAUDE.md

Treat CLAUDE.md as the place you write down what you’d otherwise re-explain. Add to it when:

- Claude makes the same mistake a second time
- A code review catches something Claude should have known about this codebase
- You type the same correction or clarification into chat that you typed last session
- A new teammate would need the same context to be productive

Keep it to facts Claude should hold in every session: build commands, conventions, project layout, “always do X” rules. If an entry is a multi-step procedure or only matters for one part of the codebase, move it to a [skill](#) or a [path-scoped rule](#) instead. The [extension overview](#) covers when to use each mechanism.

Choose where to put CLAUDE.md files

CLAUDE.md files can live in several locations, each with a different scope. The table below lists them in load order, from broadest scope to most specific, so a project instruction appears in context after a user instruction.

Scope	Location	Purpose	Use case examples	Shared with
Managed policy	• macOS: /Library/Applicat ion	Organization-wide instructions managed by	Company coding standards, security policies, compliance	All users in organization

Scope	Location	Purpose	Use case examples	Shared with
	<code>Support/ClaudeCode</code> <code>e/CLAUDE.md</code> - Linux and WSL: <code>/etc/claude-</code> <code>code/CLAUDE.md</code> - Windows: <code>C:\Program</code> <code>Files\ClaudeCode\</code> <code>CLAUDE.md</code>	IT/DevOps	requirements	
User instructions	<code>~/.claude/CLAUDE.</code> <code>md</code>	Personal preferences for all projects	Code styling preferences, personal tooling shortcuts	Just you (all projects)
Project instructions	<code>./CLAUDE.md</code> or <code>./.claude/CLAUDE.</code> <code>md</code>	Team-shared instructions for the project	Project architecture, coding standards, common workflows	Team members via source control
Local instructions	<code>./CLAUDE.local.m</code> <code>d</code>	Personal project-specific preferences; add to <code>.gitignore</code>	Your sandbox URLs, preferred test data	Just you (current project)

CLAUDE.md and CLAUDE.local.md files in the directory hierarchy above the working directory are loaded in full at launch. Files in subdirectories load on demand when Claude reads files in those directories. See [How CLAUDE.md files load](#) for the full resolution order.

For large projects, you can break instructions into topic-specific files using [project rules](#). Rules let you scope instructions to specific file types or subdirectories.

Set up a project CLAUDE.md

A project CLAUDE.md can be stored in either `./CLAUDE.md` or `./.claude/CLAUDE.md`. Create this file and add instructions that apply to anyone working on the project: build and test commands, coding standards, architectural decisions, naming conventions, and common workflows. These instructions are shared with your team through version control, so focus on project-level standards rather than personal preferences.

💡 Run `/init` to generate a starting `CLAUDE.md` automatically. Claude analyzes your codebase and creates a file with build commands, test instructions, and project conventions it discovers. If a `CLAUDE.md` already exists, `/init` suggests improvements rather than overwriting it. Refine from there with instructions Claude wouldn't discover on its own.

Set `CLAUDE_CODE_NEW_INIT=1` to enable an interactive multi-phase flow. `/init` asks which artifacts to set up: `CLAUDE.md` files, skills, and hooks. It then explores your codebase with a subagent, fills in gaps via follow-up questions, and presents a reviewable proposal before writing any files.

Write effective instructions

`CLAUDE.md` files are loaded into the context window at the start of every session, consuming tokens alongside your conversation. The [context window visualization](#) shows where `CLAUDE.md` loads relative to the rest of the startup context. Because they're context rather than enforced configuration, how you write instructions affects how reliably Claude follows them. Specific, concise, well-structured instructions work best.

Size: target under 200 lines per `CLAUDE.md` file. Longer files consume more context and reduce adherence. If your instructions are growing large, use [path-scoped rules](#) so instructions load only when Claude works with matching files. You can also split content into [imports](#) for organization, though imported files still load and enter the context window at launch.

Structure: use markdown headers and bullets to group related instructions. Claude scans structure the same way readers do: organized sections are easier to follow than dense paragraphs.

Specificity: write instructions that are concrete enough to verify. For example:

- “Use 2-space indentation” instead of “Format code properly”
- “Run `npm test` before committing” instead of “Test your changes”
- “API handlers live in `src/api/handlers/`” instead of “Keep files organized”

Consistency: if two rules contradict each other, Claude may pick one arbitrarily. Review your `CLAUDE.md` files, nested `CLAUDE.md` files in subdirectories, and [.claude/rules/](#) periodically to remove outdated or conflicting instructions. In monorepos, use [claudeMdExcludes](#) to skip `CLAUDE.md` files from other teams that aren't relevant to your work.

Import additional files

`CLAUDE.md` files can import additional files using `@path/to/import` syntax. Imported files are expanded and loaded into context at launch alongside the `CLAUDE.md` that references them.

Both relative and absolute paths are allowed. Relative paths resolve relative to the file containing the import, not the working directory. Imported files can recursively import other files, with a maximum depth of four hops.

Import parsing skips Markdown code spans and fenced code blocks. To mention a path in your `CLAUDE.md` without importing it, wrap it in backticks: writing ``@README`` keeps the text literal, while `@README` outside backticks imports the file.

To pull in a README, package.json, and a workflow guide, reference them with `@` syntax anywhere in your `CLAUDE.md`:

```
See @README for project overview and @package.json for available
npm commands for this project.
```

```
# Additional Instructions
- git workflow @docs/git-instructions.md
```

For private per-project preferences that shouldn't be checked into version control, create a `CLAUDE.local.md` at the project root. It loads alongside `CLAUDE.md` and is treated the same way. Add `CLAUDE.local.md` to your `.gitignore` so it isn't committed; running `/init` and choosing the personal option does this for you.

If you work across multiple git worktrees of the same repository, a gitignored `CLAUDE.local.md` only exists in the worktree where you created it. To share personal instructions across worktrees, import a file from your home directory instead:

```
# Individual Preferences
- @~/ .claude/my-project-instructions.md
```

 The first time Claude Code encounters external imports in a project, it shows an approval dialog listing the files. If you decline, the imports stay disabled and the dialog does not appear again.

For a more structured approach to organizing instructions, see [`.claude/rules/`](#).

AGENTS.md

Claude Code reads `CLAUDE.md`, not `AGENTS.md`. If your repository already uses `AGENTS.md` for other coding agents, create a `CLAUDE.md` that imports it so both tools read the same instructions

without duplicating them. You can also add Claude-specific instructions below the import. Claude loads the imported file at session start, then appends the rest:

```
CLAUDE.md
```

```
@AGENTS.md
```

```
## Claude Code
```

```
Use plan mode for changes under `src/billing/`.
```

A symlink also works if you don't need to add Claude-specific content:

```
ln -s AGENTS.md CLAUDE.md
```

On Windows, creating a symlink requires Administrator privileges or Developer Mode, so use the `@AGENTS.md` import instead.

Running `/init` in a repo that already has an `AGENTS.md` reads it and incorporates the relevant parts into the generated `CLAUDE.md`. It also reads other tool configs like `.cursorrules`, `.devin/rules/`, and `.windsurfrules`.

How CLAUDE.md files load

Claude Code reads `CLAUDE.md` files by walking up the directory tree from your current working directory, checking each directory along the way for `CLAUDE.md` and `CLAUDE.local.md` files. This means if you run Claude Code in `foo/bar/`, it loads instructions from `foo/bar/CLAUDE.md`, `foo/CLAUDE.md`, and any `CLAUDE.local.md` files alongside them.

All discovered files are concatenated into context rather than overriding each other. Across the directory tree, content is ordered from the filesystem root down to your working directory. For the `foo/bar/` example, `foo/CLAUDE.md` appears in context before `foo/bar/CLAUDE.md`, so instructions closer to where you launched Claude are read last. Within each directory, `CLAUDE.local.md` is appended after `CLAUDE.md`, so your personal notes are the last thing Claude reads at that level.

Claude also discovers `CLAUDE.md` and `CLAUDE.local.md` files in subdirectories under your current working directory. Instead of loading them at launch, they are included when Claude reads files in those subdirectories.

If you work in a large monorepo where other teams' CLAUDE.md files get picked up, use `claudeMdExcludes` to skip them. For the full layout of root and per-directory CLAUDE.md files and rules, see [Monorepos and large repos](#).

Block-level HTML comments (`<!-- maintainer notes -->`) in CLAUDE.md files are stripped before the content is injected into Claude's context. Use them to leave notes for human maintainers without spending context tokens on them. Comments inside code blocks are preserved. When you open a CLAUDE.md file directly with the Read tool, comments remain visible.

Load from additional directories

The `--add-dir` flag gives Claude access to additional directories outside your main working directory. By default, CLAUDE.md files from these directories are not loaded.

To also load memory files from additional directories, set the

`CLAUDE_CODE_ADDITIONAL_DIRECTORIES_CLAUDE_MD` environment variable:

```
CLAUDE_CODE_ADDITIONAL_DIRECTORIES_CLAUDE_MD=1 claude --add-dir
../shared-config
```

This loads `CLAUDE.md` , `.claude/CLAUDE.md` , `.claude/rules/*.md` , and `CLAUDE.local.md` from the additional directory. `CLAUDE.local.md` is skipped if you exclude `local` from `--setting-sources` .

Organize rules with `.claude/rules/`

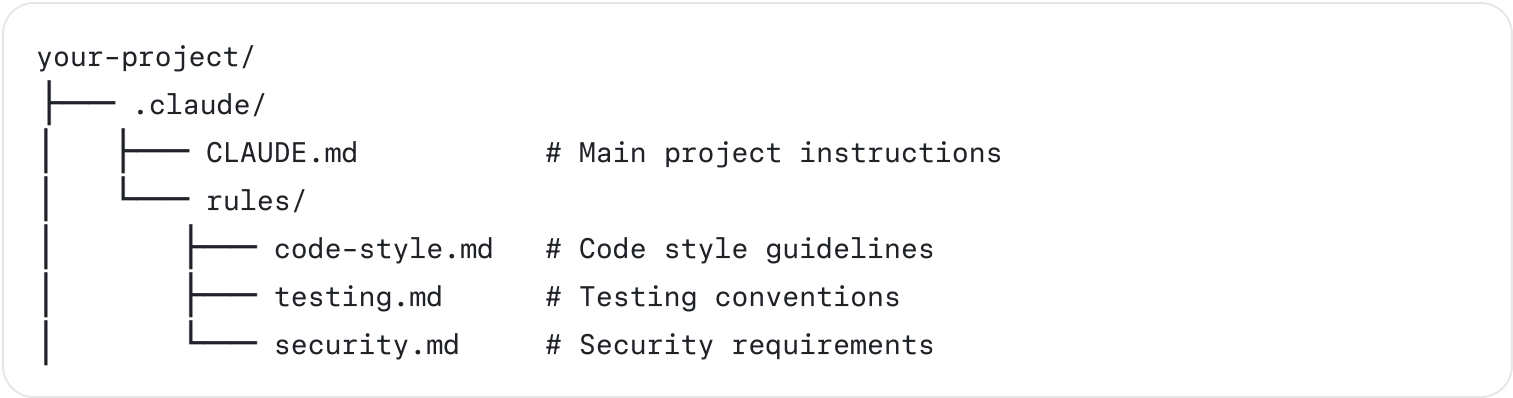
For larger projects, you can organize instructions into multiple files using the `.claude/rules/` directory. This keeps instructions modular and easier for teams to maintain. Rules can also be **scoped to specific file paths**, so they only load into context when Claude works with matching files, reducing noise and saving context space.

❗ Rules load into context every session or when matching files are opened. For task-specific instructions that don't need to be in context all the time, use **skills** instead, which only load when you invoke them or when Claude determines they're relevant to your prompt.

Set up rules

Place markdown files in your project's `.claude/rules/` directory. Each file should cover one topic, with a descriptive filename like `testing.md` or `api-design.md` . All `.md` files are discovered

recursively, so you can organize rules into subdirectories like `frontend/` or `backend/` :



Rules without `paths` **frontmatter** are loaded at launch with the same priority as `.claude/CLAUDE.md` .

Path-specific rules

Rules can be scoped to specific files using YAML frontmatter with the `paths` field. These conditional rules only apply when Claude is working with files matching the specified patterns.

```
---
paths:
  - "src/api/**/*.ts"
---

# API Development Rules

- All API endpoints must include input validation
- Use the standard error response format
- Include OpenAPI documentation comments
```

Rules without a `paths` field are loaded unconditionally and apply to all files. Path-scoped rules trigger when Claude reads files matching the pattern, not on every tool use.

Use glob patterns in the `paths` field to match files by extension, directory, or any combination:

Pattern	Matches
<code>**/*.ts</code>	All TypeScript files in any directory
<code>src/**/*.*</code>	All files under <code>src/</code> directory
<code>*.md</code>	Markdown files in the project root

Pattern	Matches
<code>src/components/*.tsx</code>	React components in a specific directory

You can specify multiple patterns and use brace expansion to match multiple extensions in one pattern:

```
---
paths:
  - "src/**/*.{ts,tsx}"
  - "lib/**/*.ts"
  - "tests/**/*.test.ts"
---
```

Share rules across projects with symlinks

The `.claude/rules/` directory supports symlinks, so you can maintain a shared set of rules and link them into multiple projects. Symlinks are resolved and loaded normally, and circular symlinks are detected and handled gracefully.

This example links both a shared directory and an individual file:

```
ln -s ~/shared-claude-rules .claude/rules/shared
ln -s ~/company-standards/security.md .claude/rules/security.md
```

User-level rules

Personal rules in `~/.claude/rules/` apply to every project on your machine. Use them for preferences that aren't project-specific:

```
~/.claude/rules/
├── preferences.md    # Your personal coding preferences
└── workflows.md     # Your preferred workflows
```

User-level rules are loaded before project rules, giving project rules higher priority.

Manage CLAUDE.md for large teams

For organizations deploying Claude Code across teams, you can centralize instructions and control

which CLAUDE.md files are loaded.

Deploy organization-wide CLAUDE.md

Organizations can deploy a centrally managed CLAUDE.md that applies to all users on a machine. This file cannot be excluded by individual settings.

1 Create the file at the managed policy location

- macOS: `/Library/Application Support/ClaudeCode/CLAUDE.md`
- Linux and WSL: `/etc/claude-code/CLAUDE.md`
- Windows: `C:\Program Files\ClaudeCode\CLAUDE.md`

2 Deploy with your configuration management system

Use MDM, Group Policy, Ansible, or similar tools to distribute the file across developer machines. See [managed settings](#) for other organization-wide configuration options.

The `claudeMd` key lets you put managed CLAUDE.md content directly inside `managed-settings.json` instead of deploying a separate file.

Scope: every Claude Code session on the machine, in every repository. For repository-specific guidance, commit a project CLAUDE.md instead.

Precedence: same as a managed CLAUDE.md file. Loads before user and project CLAUDE.md.

Where it's honored: managed and policy settings only. Setting `claudeMd` in user, project, or local settings has no effect.

The example below adds behavioral instructions directly in a managed settings file:

```
{
  "claudeMd": "Always run `make lint` before committing.\nNever\npush directly to main."
}
```

A managed CLAUDE.md and [managed settings](#) serve different purposes. Use settings for technical enforcement and CLAUDE.md for behavioral guidance:

Concern	Configure in
Block specific tools, commands, or file paths	Managed settings: <code>permissions.deny</code>
Enforce sandbox isolation	Managed settings: <code>sandbox.enabled</code>
Environment variables and API provider routing	Managed settings: <code>env</code>
Authentication method and organization lock	Managed settings: <code>forceLoginMethod</code> , <code>forceLoginOrgUUID</code>
Code style and quality guidelines	Managed CLAUDE.md
Data handling and compliance reminders	Managed CLAUDE.md
Behavioral instructions for Claude	Managed CLAUDE.md

Settings rules are enforced by the client regardless of what Claude decides to do. CLAUDE.md instructions shape Claude’s behavior but are not a hard enforcement layer.

Exclude specific CLAUDE.md files

In large monorepos, ancestor CLAUDE.md files may contain instructions that aren’t relevant to your work. The `claudeMdExcludes` setting lets you skip specific files by path or glob pattern.

This example excludes a top-level CLAUDE.md and a rules directory from a parent folder. Add it to `.claude/settings.local.json` so the exclusion stays local to your machine:

```
{
  "claudeMdExcludes": [
    "**/monorepo/CLAUDE.md",
    "/home/user/monorepo/other-team/.claude/rules/**"
  ]
}
```

Patterns are matched against absolute file paths using glob syntax. You can configure `claudeMdExcludes` at any settings layer: user, project, local, or managed policy. Arrays merge across layers.

Managed policy CLAUDE.md files cannot be excluded. This ensures organization-wide instructions always apply regardless of individual settings.

Auto memory

Auto memory lets Claude accumulate knowledge across sessions without you writing anything. Claude saves notes for itself as it works: build commands, debugging insights, architecture notes, code style preferences, and workflow habits. Claude doesn't save something every session. It decides what's worth remembering based on whether the information would be useful in a future conversation.

❗ Auto memory requires Claude Code v2.1.59 or later. Check your version with `claude --version`.

Enable or disable auto memory

Auto memory is on by default. To toggle it, open `/memory` in a session and use the auto memory toggle, or set `autoMemoryEnabled` in your project settings:

```
{
  "autoMemoryEnabled": false
}
```

To disable auto memory via environment variable, set `CLAUDE_CODE_DISABLE_AUTO_MEMORY=1`.

Storage location

Each project gets its own memory directory at `~/.claude/projects/<project>/memory/`. The `<project>` path is derived from the git repository, so all worktrees and subdirectories within the same repo share one auto memory directory. Outside a git repo, the project root is used instead.

To store auto memory in a different location, set `autoMemoryDirectory` in your `settings.json`. It is read from any **settings scope**: user, project, local, policy, or `--settings`.

```
{
  "autoMemoryDirectory": "~/my-custom-memory-dir"
}
```

The value must be an absolute path or start with `~/`. When set in a project's `.claude/settings.json` or `.claude/settings.local.json`, the value is honored only after you accept the workspace trust dialog for that folder, the same gate that governs hooks.

The directory contains a `MEMORY.md` entrypoint and optional topic files:

```
~/.claude/projects/<project>/memory/  
├── MEMORY.md          # Concise index, loaded into every  
session  
├── debugging.md       # Detailed notes on debugging patterns  
├── api-conventions.md # API design decisions  
└── ...                # Any other topic files Claude creates
```

`MEMORY.md` acts as an index of the memory directory. Claude reads and writes files in this directory throughout your session, using `MEMORY.md` to keep track of what's stored where.

Auto memory is machine-local. All worktrees and subdirectories within the same git repository share one auto memory directory. Files are not shared across machines or cloud environments.

How it works

The first 200 lines of `MEMORY.md`, or the first 25KB, whichever comes first, are loaded at the start of every conversation. Content beyond that threshold is not loaded at session start. Claude keeps `MEMORY.md` concise by moving detailed notes into separate topic files.

This limit applies only to `MEMORY.md`. `CLAUDE.md` files are loaded in full regardless of length, though shorter files produce better adherence.

Topic files like `debugging.md` or `patterns.md` are not loaded at startup. Claude reads them on demand using its standard file tools when it needs the information.

Claude reads and writes memory files during your session. When you see “Writing memory” or “Recalled memory” in the Claude Code interface, Claude is actively updating or reading from `~/.claude/projects/<project>/memory/`.

Audit and edit your memory

Auto memory files are plain markdown you can edit or delete at any time. Run `/memory` to browse and open memory files from within a session.

View and edit with `/memory`

The `/memory` command lists all `CLAUDE.md`, `CLAUDE.local.md`, and rules files loaded in your

current session, lets you toggle auto memory on or off, and provides a link to open the auto memory folder. Select any file to open it in your editor.

When you ask Claude to remember something, like “always use pnpm, not npm” or “remember that the API tests require a local Redis instance,” Claude saves it to auto memory. To add instructions to CLAUDE.md instead, ask Claude directly, like “add this to CLAUDE.md,” or edit the file yourself via `/memory` .

Troubleshoot memory issues

These are the most common issues with CLAUDE.md and auto memory, along with steps to debug them.

Claude isn't following my CLAUDE.md

CLAUDE.md content is delivered as a user message after the system prompt, not as part of the system prompt itself. Claude reads it and tries to follow it, but there's no guarantee of strict compliance, especially for vague or conflicting instructions.

To debug:

- Run `/memory` to verify your CLAUDE.md and CLAUDE.local.md files are being loaded. If a file isn't listed, Claude can't see it.
- Check that the relevant CLAUDE.md is in a location that gets loaded for your session (see [Choose where to put CLAUDE.md files](#)).
- Make instructions more specific. “Use 2-space indentation” works better than “format code nicely.”
- Look for conflicting instructions across CLAUDE.md files. If two files give different guidance for the same behavior, Claude may pick one arbitrarily.

If the instruction is something that must run at a specific point, such as before every commit or after each file edit, write it as a **hook** instead. Hooks execute as shell commands at fixed lifecycle events and apply regardless of what Claude decides to do.

For instructions you want at the system prompt level, use `--append-system-prompt` . This must be passed every invocation, so it's better suited to scripts and automation than interactive use.

💡 Use the `InstructionsLoaded` hook to log exactly which instruction files are loaded, when they load, and why. This is useful for debugging path-specific rules or lazy-loaded files in subdirectories.

I don't know what auto memory saved

Run `/memory` and select the auto memory folder to browse what Claude has saved. Everything is plain markdown you can read, edit, or delete.

My CLAUDE.md is too large

Files over 200 lines consume more context and may reduce adherence. Use path-scoped rules to load instructions only when Claude works with matching files, or trim content that isn't needed in every session. Splitting into `@path imports` helps organization but does not reduce context, since imported files load at launch.

Instructions seem lost after `/compact`

Project-root CLAUDE.md survives compaction: after `/compact`, Claude re-reads it from disk and re-injects it into the session. Nested CLAUDE.md files in subdirectories are not re-injected automatically; they reload the next time Claude reads a file in that subdirectory.

If an instruction disappeared after compaction, it was either given only in conversation or lives in a nested CLAUDE.md that hasn't reloaded yet. Add conversation-only instructions to CLAUDE.md to make them persist. See What survives compaction for the full breakdown.

See Write effective instructions for guidance on size, structure, and specificity.

Related resources

- Debug your configuration: diagnose why CLAUDE.md or settings aren't taking effect
- Skills: package repeatable workflows that load on demand
- Settings: configure Claude Code behavior with settings files
- Subagent memory: let subagents maintain their own auto memory